

CTI ADP Software: Test Plan & Test Cases

Version: 1.0 | **Date:** 2026-05-04 | **Status:** Active

PART A: TEST PLAN

1. INTRODUCTION

This document defines the test plan and detailed test cases for the CTI ADP Software suite, covering 4 projects:

- 1. E-Commerce Frontend (React/Vite)
- 2. E-Commerce Backend (Express.js/DDDD)
- 3. Mobile App (Android/Jetpack Compose)
- 4. Desktop App (JavaFX/PostgreSQL)

2. OBJECTIVES

- Validate all critical features across all 4 projects
- Ensure data consistency between frontend, backend, and databases
- Verify security (authentication, authorization, data protection)
- Test error handling and edge cases
- Achieve minimum coverage targets (90% backend, 75%+ frontend)
- Support continuous integration and deployment

3. TEST SCOPE

3.1 In Scope by Project

E-Commerce Frontend:

- CartContext state management (add/remove/update/clear)
- localStorage persistence and recovery
- Stock validation and quantity constraints
- Component rendering and user interactions
- Toast notifications

E-Commerce Backend:

- Authentication use cases (register, login, JWT validation)
- Product catalog operations (list, search, detail)
- Order creation and management
- Cart to order conversion
- API endpoint contracts
- Security middleware

Mobile App:

- WebView initialization and navigation
- Screen transitions using Jetpack Navigation
- WebView-native communication
- Compose UI component rendering
- User interaction with buttons and forms

Desktop App:

- User login/registration
- Stock management CRUD (Create, Read, Update, Delete)
- Database persistence verification
- Scene navigation
- Data validation and error handling

3.2 Out of Scope

- Performance/load testing (future phase)
- Accessibility testing (future phase)
- End-to-end browser automation (manual testing)
- Third-party library internals
- Database migration testing

4. TEST APPROACH

4.1 Testing Pyramid

- Level 3: Manual E2E Testing (10%)
Browser workflows, staging validation
- Level 2: Integration Testing (30%)
API contracts, context integration, DB operations
- Level 1: Unit Testing (60%)
Business logic, utilities, calculations

4.2 Execution Strategy

1. **Local Development:** Developers run tests before commit
2. **Pre-Commit:** Type check + lint + tests (git hook)
3. **CI Pipeline:** Automated tests on every PR
4. **Post-Merge:** Manual QA testing on staging

5. SCHEDULE

Phase	Duration	Activities
Phase 1: Setup	Week 1	Configure test environments, tools, fixtures

Phase	Duration	Activities
Phase 2: Backend	Week 2-3	Unit & integration tests for API layer
Phase 3: Frontend	Week 3-4	Context, component, and E2E tests
Phase 4: Mobile	Week 4-5	Compose UI and WebView tests
Phase 5: Desktop	Week 5-6	UI and database persistence tests
Phase 6: Integration	Week 6-7	Cross-project workflows, data sync
Phase 7: Regression	Week 7-8	Full test suite, coverage validation

Go-Live: Week 8 (after all phases complete and targets met)

6. RESOURCES

Role	Responsibility	Count
QA Lead	Test plan, coverage validation, release approval	1
Backend Dev	Backend unit/integration tests, API contracts	2
Frontend Dev	Frontend context/component tests	2
Mobile Dev	Android/Espresso tests	1
Desktop Dev	JavaFX/JUnit5 tests	1

7. RISK ASSESSMENT

Risk	Likelihood	Impact	Mitigation
Database connection failures	Medium	High	Mock DB in tests, use test database
Flaky tests (timing issues)	Medium	High	Use proper async/await, avoid fake timers
localStorage data corruption	Low	Medium	Test JSON parsing errors, add recovery logic
JWT token expiration	Low	Medium	Mock time in tests, test refresh flows
Stock quantity race conditions	Medium	High	Lock testing, atomic operations

8. PASS/FAIL CRITERIA

8.1 Test Pass Criteria

- ☐ All test cases pass (0 failures)
- ☐ Coverage meets targets: Backend 90%, Frontend 85%, Mobile 70%, Desktop 80%
- ☐ No flaky tests (0% failure rate on repeated runs)
- ☐ No type errors (`tsc --noEmit` passes)
- ☐ No linting errors (eslint/checkstyle pass)

☐ Manual QA checklist completed

8.2 Test Fail Criteria

- Any test fails in local or CI environment
- Coverage drops below targets
- New code has 0 test coverage
- Type errors detected
- Linting errors present
- Flaky tests not investigated and fixed

PART B: TEST CASES

E-COMMERCE BACKEND TEST CASES

Category 1: Authentication & Security

TC-BE-AUTH-001: User Registration with Valid Data

Type: Unit/Integration
Priority: Critical
Component: RegisterClientUseCase

Field	Value
Precondition	No user with email exists
Input Data	{ email: "newuser@test.com", password: "SecurePass123!", name: "Test User" }
Expected Result	User created, password hashed with bcrypt, success response with user ID
Assertion	expect(user.email).toBe("newuser@test.com") expect(user.passwordHash).not.toBe(password)

TC-BE-AUTH-002: User Registration with Duplicate Email

Type: Unit/Integration
Priority: High
Component: RegisterClientUseCase

Field	Value
Precondition	User "duplicate@test.com" already exists
Input Data	{ email: "duplicate@test.com", password: "NewPass123!", name: "Another User" }
Expected Result	Registration fails with "Email already registered" error
Assertion	expect(useCase.register()).rejects.toThrow("Email already registered")

TC-BE-AUTH-003: User Login with Correct Credentials

Type: Unit/Integration
Priority: Critical
Component: LoginClientUseCase

Field	Value
Precondition	User exists with email "user@test.com", password "SecurePass123!"
Input Data	{ email: "user@test.com", password: "SecurePass123!" }
Expected Result	Login success, JWT token returned with valid payload
Assertion	expect(token).toBeTruthy() expect(decoded.email).toBe("user@test.com")

TC-BE-AUTH-004: User Login with Wrong Password

Type: Unit/Integration
Priority: High
Component: LoginClientUseCase

Field	Value
Precondition	User "user@test.com" exists with password "SecurePass123!"
Input Data	{ email: "user@test.com", password: "WrongPassword" }
Expected Result	Login fails with 401 Unauthorized
Assertion	expect(useCase.login()).rejects.toThrow("Invalid credentials")

TC-BE-AUTH-005: JWT Token Validation

Type: Unit
Priority: Critical

Component: JwtService

Field	Value
Precondition	Valid JWT token exists
Input Data	Valid token, valid secret
Expected Result	Token decoded successfully, payload returned
Assertion	<code>expect(decoded.email).toBe("user@test.com")</code>

TC-BE-AUTH-006: JWT Token Validation with Expired Token

Type: Unit
Priority: High
Component: JwtService

Field	Value
Precondition	Token expired 1 hour ago
Input Data	Expired token
Expected Result	Validation fails with "Token expired" error
Assertion	<code>expect(jwtService.verify()).rejects.toThrow("Token expired")</code>

TC-BE-AUTH-007: Authentication Middleware Validation

Type: Integration
Priority: Critical
Component: authMiddleware

Field	Value
Precondition	Request to protected endpoint without token
Input Data	Headers: {} (no Authorization header)
Expected Result	Request rejected with 401 Unauthorized
Assertion	<code>expect(response.status).toBe(401)</code>

Category 2: Product Management

TC-BE-PROD-001: Get All Products

Type: Unit/Integration
Priority: High
Component: GetProductsUseCase

Field	Value
Precondition	Database contains 5 products
Input Data	No filters
Expected Result	Array of 5 products returned
Assertion	<code>expect(products.length).toBe(5)</code> <code>expect(products[0].id).toBeTruthy()</code>

TC-BE-PROD-002: Get Product by ID

Type: Unit/Integration
Priority: High
Component: GetProductByIdUseCase

Field	Value
Precondition	Product with ID 1 exists
Input Data	<code>productId: 1</code>
Expected Result	Product object returned with all fields
Assertion	<code>expect(product.id).toBe(1)</code> <code>expect(product.name).toBeTruthy()</code>

TC-BE-PROD-003: Get Product by Invalid ID

Type: Unit/Integration
Priority: Medium
Component: GetProductByIdUseCase

Field	Value
Precondition	No product with ID 999 exists
Input Data	<code>productId: 999</code>
Expected Result	404 Not Found error

Field	Value
Assertion	expect(useCase.getById()).rejects.toThrow("Product not found")

Category 3: Order Management

TC-BE-ORDER-001: Create Order with Valid Data

Type: Integration
Priority: Critical
Component: CreateOrderUseCase

Field	Value
Precondition	Products exist with sufficient stock; user authenticated
Input Data	{ clientId: 1, items: [{ productId: 1, quantity: 2 }], totalAmount: 199.98 }
Expected Result	Order created, ID returned, stock updated
Assertion	expect(order.id).toBeTruthy() expect(order.status).toBe("pending")

TC-BE-ORDER-002: Create Order with Insufficient Stock

Type: Integration
Priority: High
Component: CreateOrderUseCase

Field	Value
Precondition	Product ID 1 has only 2 units in stock
Input Data	{ items: [{ productId: 1, quantity: 10 }] }
Expected Result	Order creation fails, "Insufficient stock" error
Assertion	expect(useCase.create()).rejects.toThrow("Insufficient stock")

TC-BE-ORDER-003: Create Order with Invalid Product

Type: Integration
Priority: High
Component: CreateOrderUseCase

Field	Value
Precondition	Product ID 999 does not exist
Input Data	{ items: [{ productId: 999, quantity: 1 }] }
Expected Result	Order creation fails, "Product not found" error
Assertion	expect(useCase.create()).rejects.toThrow("Product not found")

TC-BE-ORDER-004: Create Order with Empty Items

Type: Unit
Priority: Medium
Component: CreateOrderUseCase

Field	Value
Precondition	N/A
Input Data	{ clientId: 1, items: [] }
Expected Result	Validation error, "Order must contain at least one item"
Assertion	expect(useCase.create()).rejects.toThrow("Items required")

E-COMMERCE FRONTEND TEST CASES

Category 1: CartContext Management

TC-FE-CART-001: Add Product to Empty Cart

Type: Unit
Priority: Critical
Component: CartContext

Field	Value
Precondition	Cart is empty
Input Data	addToCart(mockProduct, 1) where mockProduct = { id: 1, onhand: 10, price: 99.99 }
Expected Result	Cart contains 1 item, totalItems = 1, totalAmount = 99.99
Assertion	expect(items.length).toBe(1) expect(totalItems).toBe(1) expect(totalAmount).toBe(99.99)

TC-FE-CART-002: Add Product Already in Cart

Type: Unit
Priority: High
Component: CartContext

Field	Value
Precondition	Cart contains product ID 1, quantity 2
Input Data	addToCart(sameProduct, 3)
Expected Result	Quantity updated to 5 (2+3), no duplicate item
Assertion	expect(items.length).toBe(1) expect(items[0].quantity).toBe(5)

TC-FE-CART-003: Add Product Exceeding Stock

Type: Unit
Priority: High
Component: CartContext

Field	Value
Precondition	Cart contains product, onhand = 10
Input Data	addToCart(product, 15)
Expected Result	Quantity capped at 10, toast shown "Only 10 available in stock"
Assertion	expect(items[0].quantity).toBe(10) expect(mockToast).toHaveBeenCalledWith(expect.objectContaining({ title: "Stock Limit Reached" })))

TC-FE-CART-004: Update Quantity to Valid Value

Type: Unit
Priority: High
Component: CartContext

Field	Value
Precondition	Cart contains product ID 1, quantity 2
Input Data	updateQuantity(1, 5)

Field	Value
Expected Result	Quantity updated to 5, totalAmount recalculated
Assertion	expect(items[0].quantity).toBe(5) expect(totalAmount).toBe(expectedNewAmount)

TC-FE-CART-005: Update Quantity Exceeding Stock

Type: Unit
Priority: High
Component: CartContext

Field	Value
Precondition	Product onhand = 8, current quantity = 3
Input Data	updateQuantity(1, 15)
Expected Result	Quantity capped at 8, toast shown
Assertion	expect(items[0].quantity).toBe(8) expect(mockToast).toHaveBeenCalled()

TC-FE-CART-006: Update Quantity to Zero

Type: Unit
Priority: High
Component: CartContext

Field	Value
Precondition	Cart contains 2 items
Input Data	updateQuantity(1, 0)
Expected Result	Item removed from cart
Assertion	expect(items.length).toBe(1) expect(items[0].product.id).not.toBe(1)

TC-FE-CART-007: Remove Product from Cart

Type: Unit
Priority: High
Component: CartContext

Field	Value
Precondition	Cart contains 2 items
Input Data	removeFromCart(1)
Expected Result	Product ID 1 removed, 1 item remains
Assertion	expect(items.length).toBe(1) expect(items[0].product.id).toBe(2)

TC-FE-CART-008: Clear Cart

Type: Unit
Priority: Medium
Component: CartContext

Field	Value
Precondition	Cart contains 3 items
Input Data	clearCart()
Expected Result	All items removed, localStorage cleared
Assertion	expect(items.length).toBe(0) expect(localStorage.getItem("lpa_cart")).toBeNull()

Category 2: localStorage Persistence

TC-FE-STORE-001: Cart Persists to localStorage

Type: Integration
Priority: High
Component: CartContext

Field	Value
Precondition	CartProvider mounted
Input Data	addToCart(product, 2)
Expected Result	localStorage "lpa_cart" contains cart JSON
Assertion	expect(localStorage.getItem("lpa_cart")).toBeTruthy() expect(JSON.parse(localStorage.getItem("lpa_cart")).length).toBe(1)

TC-FE-STORE-002: Cart Loads from localStorage on Mount

Type: Integration
Priority: High
Component: CartContext

Field	Value
Precondition	localStorage contains valid cart JSON
Input Data	New CartProvider mounted
Expected Result	Cart loaded from localStorage
Assertion	<code>expect(items.length).toBe(previousItemCount)</code>

TC-FE-STORE-003: Handle Corrupted localStorage

Type: Integration
Priority: Medium
Component: CartContext

Field	Value
Precondition	localStorage contains invalid JSON: <code>"{ invalid json"</code>
Input Data	CartProvider mounted
Expected Result	Cart initializes as empty (error suppressed)
Assertion	<code>expect(items.length).toBe(0)</code> (no error thrown)

Category 3: Component Rendering

TC-FE-COMP-001: AddToCartButton Renders

Type: Component
Priority: Medium
Component: AddToCartButton

Field	Value
Precondition	Component rendered with product prop
Input Data	<code><AddToCartButton product={mockProduct} /></code>
Expected Result	Button displayed with "Add to Cart" text

Field	Value
Assertion	expect(screen.getByText("Add to Cart")).toBeInTheDocument()

TC-FE-COMP-002: AddToCartButton Click

Type: Component
Priority: High
Component: AddToCartButton

Field	Value
Precondition	CartProvider wrapping component
Input Data	User clicks "Add to Cart" button
Expected Result	Product added to cart, confirmation shown
Assertion	expect(addToCart).toHaveBeenCalledTimes(mockProduct, 1)

MOBILE APP TEST CASES

Category 1: WebView & Navigation

TC-MOBILE-WV-001: WebView Loads Successfully

Type: Component
Priority: Critical
Component: MainActivity WebView

Field	Value
Precondition	App launched, internet available
Input Data	WebView URL: https://ecommerce.local
Expected Result	WebView loads and displays content
Assertion	onPageFinished callback triggered, no errors

TC-MOBILE-WV-002: Navigation Between Screens

Type: Component
Priority: High
Component: Navigation

Field	Value
Precondition	App on Home screen
Input Data	User taps "Products" navigation item
Expected Result	Navigation to Products screen, WebView updates
Assertion	Current route = "/products", WebView URL updated

TC-MOBILE-WV-003: WebView Communication

Type: Integration
Priority: High
Component: WebViewClient

Field	Value
Precondition	WebView loaded with cart functionality
Input Data	JavaScript calls <code>addToCart(productId, quantity)</code>
Expected Result	Native app receives intent, performs action
Assertion	Bridge method executed, data passed correctly

Category 2: Compose UI Components

TC-MOBILE-UI-001: Product Card Renders

Type: Component
Priority: Medium
Component: ProductCard

Field	Value
Precondition	Component rendered with product data
Input Data	<code>ProductCard(product = mockProduct)</code>
Expected Result	Product name, price, image displayed

Field	Value
Assertion	assertEquals(mockProduct.name, captured_text)

TC-MOBILE-UI-002: Button Click Handling

Type: Component
Priority: High
Component: CartButton

Field	Value
Precondition	Button rendered in Compose
Input Data	User taps button
Expected Result	onClick callback triggered
Assertion	verify(mockCallback, times(1)).invoke()

DESKTOP APP TEST CASES

Category 1: Authentication

TC-DESK-AUTH-001: Login with Valid Credentials

Type: Integration
Priority: Critical
Component: LoginController

Field	Value
Precondition	User exists in database
Input Data	username: "admin", password: "AdminPass123!"
Expected Result	Login succeeds, main scene displayed
Assertion	assertTrue(user.isAuthenticated()) assertEquals("admin", currentUser.getUsername())

TC-DESK-AUTH-002: Login with Wrong Password

Type: Integration
Priority: High
Component: LoginController

Field	Value
Precondition	User "admin" exists
Input Data	username: "admin", password: "WrongPass"
Expected Result	Login fails, error message shown
Assertion	assertFalse(user.isAuthenticated()) assertNotNull(errorMessage)

Category 2: Stock Management CRUD

TC-DESK-STOCK-001: Create New Product

Type: Integration
Priority: High
Component: StockController

Field	Value
Precondition	Database connected, unique SKU provided
Input Data	{ sku: "NEW-001", name: "New Product", quantity: 50, price: 49.99 }
Expected Result	Product created, ID returned, database updated
Assertion	assertNotNull(productId) assertEquals(50, database.getProduct(productId).getQuantity())

TC-DESK-STOCK-002: Read Product

Type: Unit
Priority: Medium
Component: ProductRepository

Field	Value
Precondition	Product exists in database
Input Data	productId = 1
Expected Result	Product details retrieved

Field	Value
Assertion	assertNotNull(product) assertEquals("Product Name", product.getName())

TC-DESK-STOCK-003: Update Product Quantity

Type: Integration
Priority: High
Component: StockController

Field	Value
Precondition	Product exists with quantity 50
Input Data	updateQuantity(productId, 75)
Expected Result	Quantity updated in database
Assertion	assertEquals(75, database.getProduct(productId).getQuantity())

TC-DESK-STOCK-004: Delete Product

Type: Integration
Priority: Medium
Component: StockController

Field	Value
Precondition	Product exists with no active orders
Input Data	deleteProduct(productId)
Expected Result	Product removed from database
Assertion	assertNull(database.getProduct(productId))

TC-DESK-STOCK-005: Delete Product with Active Orders

Type: Integration
Priority: High
Component: StockController

Field	Value
Precondition	Product has active orders

Field	Value
Input Data	deleteProduct(productId)
Expected Result	Deletion fails, "Cannot delete product with active orders" error
Assertion	assertThrows(BusinessException.class, () -> controller.delete(productId))

Category 3: Database Persistence

TC-DESK-DB-001: HikariCP Connection Pool Works

Type: Integration
Priority: Critical
Component: DatabaseConnection

Field	Value
Precondition	PostgreSQL database running
Input Data	Connection attempt
Expected Result	Connection established, queries execute
Assertion	assertTrue(pool.getConnections().size() > 0)

TC-DESK-DB-002: Data Persists After Restart

Type: Integration
Priority: High
Component: ProductRepository

Field	Value
Precondition	Product created and saved
Input Data	Simulate application restart, query database
Expected Result	Product still exists with same data
Assertion	assertEquals(originalProduct, queriedProduct)

CROSS-PROJECT TEST CASES

TC-CROSS-001: Order Flow from Frontend to Backend to Desktop

Type: Integration/E2E
Priority: Critical

Step	Action	Expected Result
1	Frontend: Add product to cart via CartContext	Cart updated locally
2	Frontend: Submit checkout form with cart data	POST /api/orders sent to backend
3	Backend: CreateOrderUseCase receives request	Order validation passes
4	Backend: Prisma saves order to database	Database contains new order record
5	Desktop: Query database for new orders	New order appears in desktop app
6	Desktop: Update stock quantities	Stock decremented correctly
7	Verify: All data consistent across systems	No discrepancies

TC-CROSS-002: Authentication Across Mobile and Backend

Type: Integration
Priority: High

Step	Action	Expected Result
1	Mobile: User enters credentials in login form	Input captured
2	Mobile: User taps Login button	HTTP request sent to backend
3	Backend: LoginClientUseCase processes credentials	Password verified via bcrypt
4	Backend: JWT token generated and returned	Token sent to mobile app
5	Mobile: Store token in secure storage	Token persisted
6	Mobile: Use token for subsequent API calls	Requests authenticated
7	Backend: authMiddleware validates token	Requests proceed

TEST EXECUTION MATRIX

Project	Unit	Integration	Component	Manual	Coverage Target
Backend	45	20	-	5	90%+
Frontend	35	15	10	5	85%+
Mobile	10	8	12	5	70%+
Desktop	25	15	10	5	80%+
TOTAL	115	58	32	20	-

TEST COMPLETION CHECKLIST

- ☐ All unit tests written and passing
- ☐ All integration tests written and passing
- ☐ All component tests written and passing
- ☐ Coverage reports generated and reviewed
- ☐ Cross-project workflows tested
- ☐ Manual QA checklists completed
- ☐ Performance benchmarks verified
- ☐ No flaky tests identified
- ☐ Documentation updated
- ☐ Sign-off from QA Lead

Document Owner: QA Lead

Last Updated: 2026-05-04

Next Review: 2026-06-04